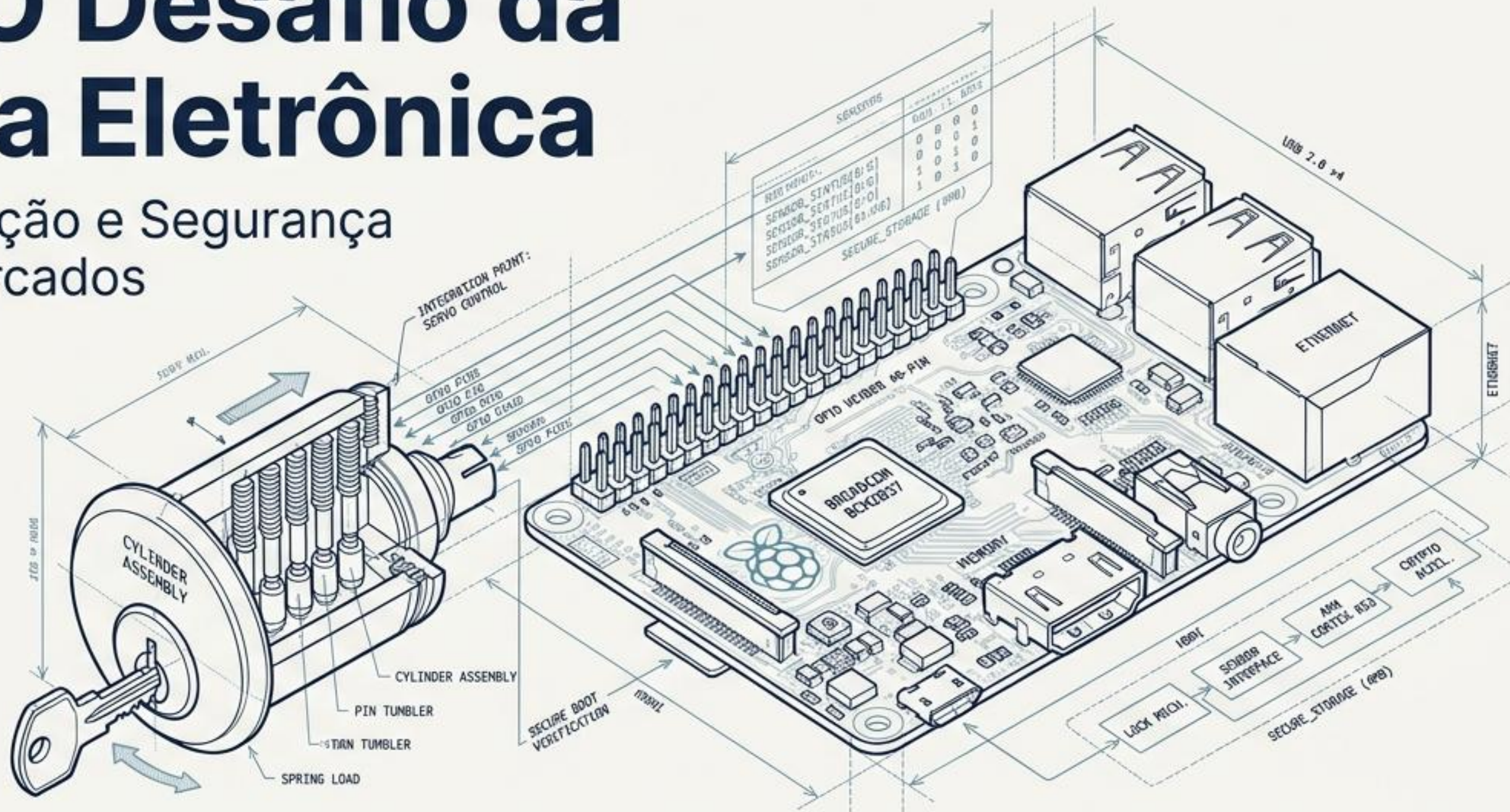
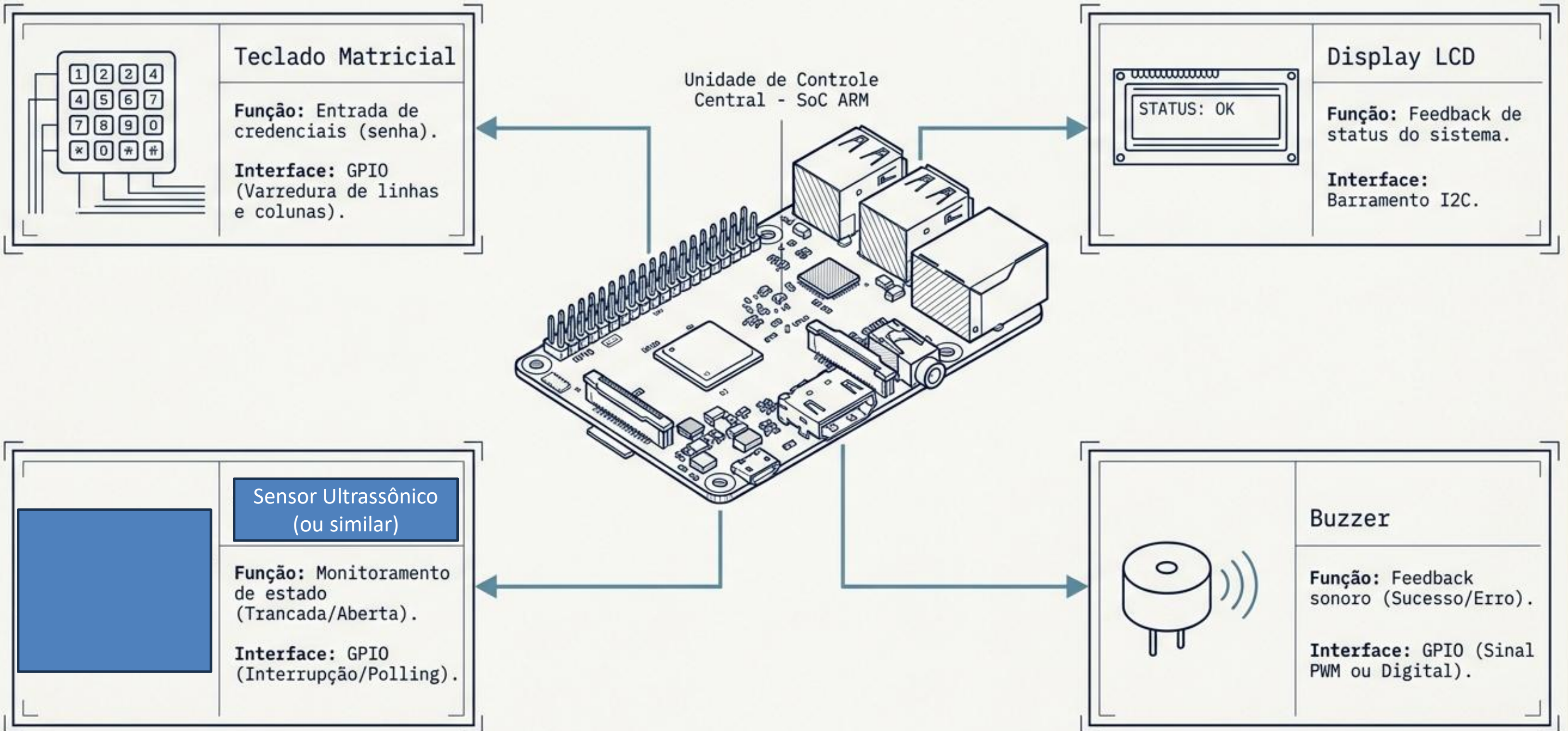


Aula 10: O Desafio da Fechadura Eletrônica

Arquitetura, Integração e Segurança em Sistemas Embarcados (Raspberry Pi 3)



Parâmetros da Missão: Hardware e Interfaces



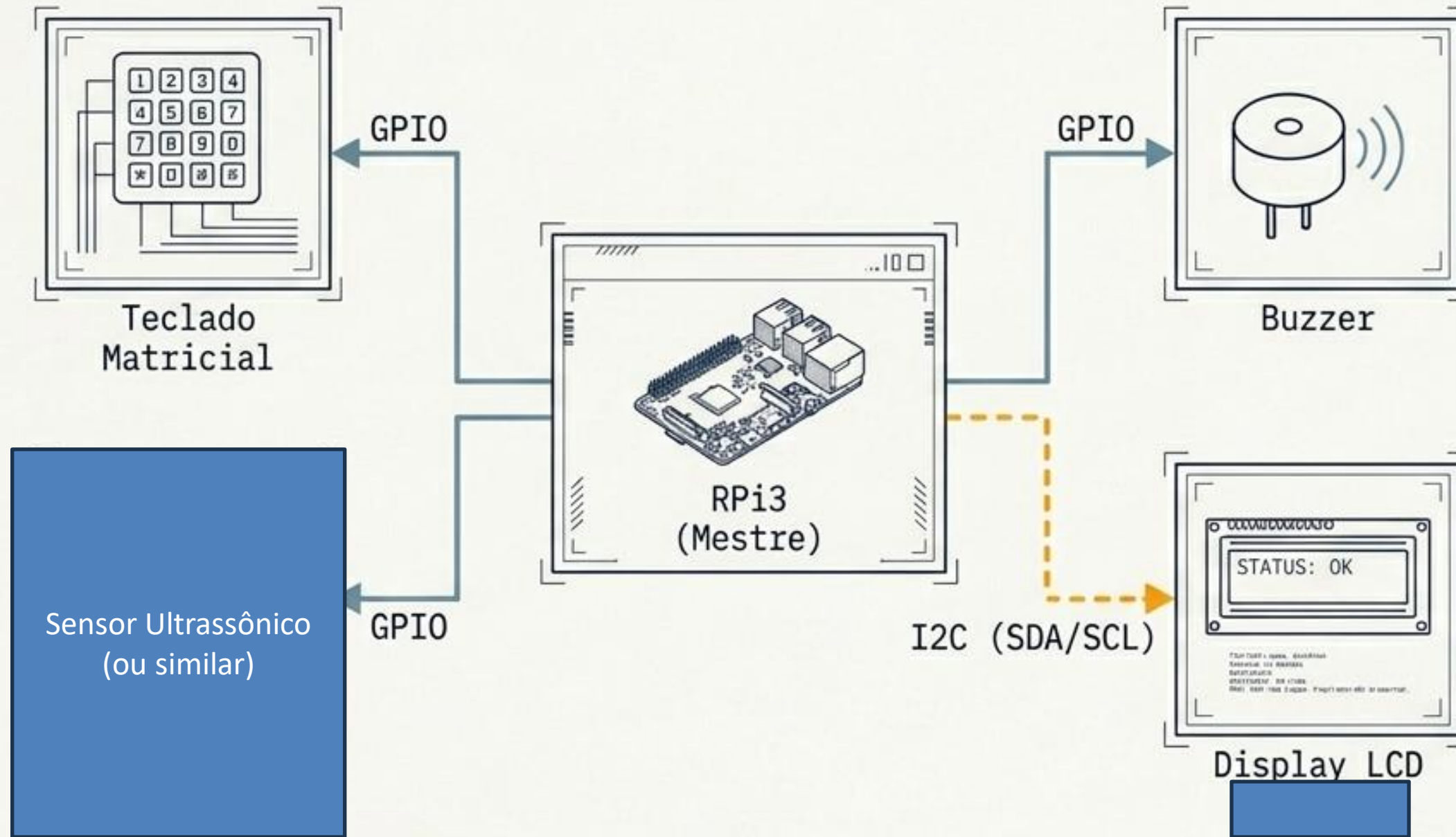
Matriz de Validação: Requisitos e Casos de Teste

Requisito	Teste	Esperado (Telemetria)
RF1: O sistema deve registrar senhas numéricas via teclado.	Inserção de sequência limite (ex: 4 a 6 dígitos), incluindo backspace.	Captura exata sem bouncing de teclas.
RF2: O LCD deve exibir o status em tempo real.	Transição de estado bloqueado para desbloqueado.	Atualização da tela em < 200ms após validação.
RF3: O sensor deve verificar integridade física da tranca.	Abertura forçada da porta (simulação de curto).	Disparo de alerta no LCD e Buzzer.
RNF1 (Confiabilidade): Recuperação de erros de entrada.	Múltiplas entradas incorretas seguidas.	Bloqueio temporário (cooldown) sem travamento do S0.



Registre todas as capturas de tela dos testes isolados (ex: simuladores) como evidência técnica.

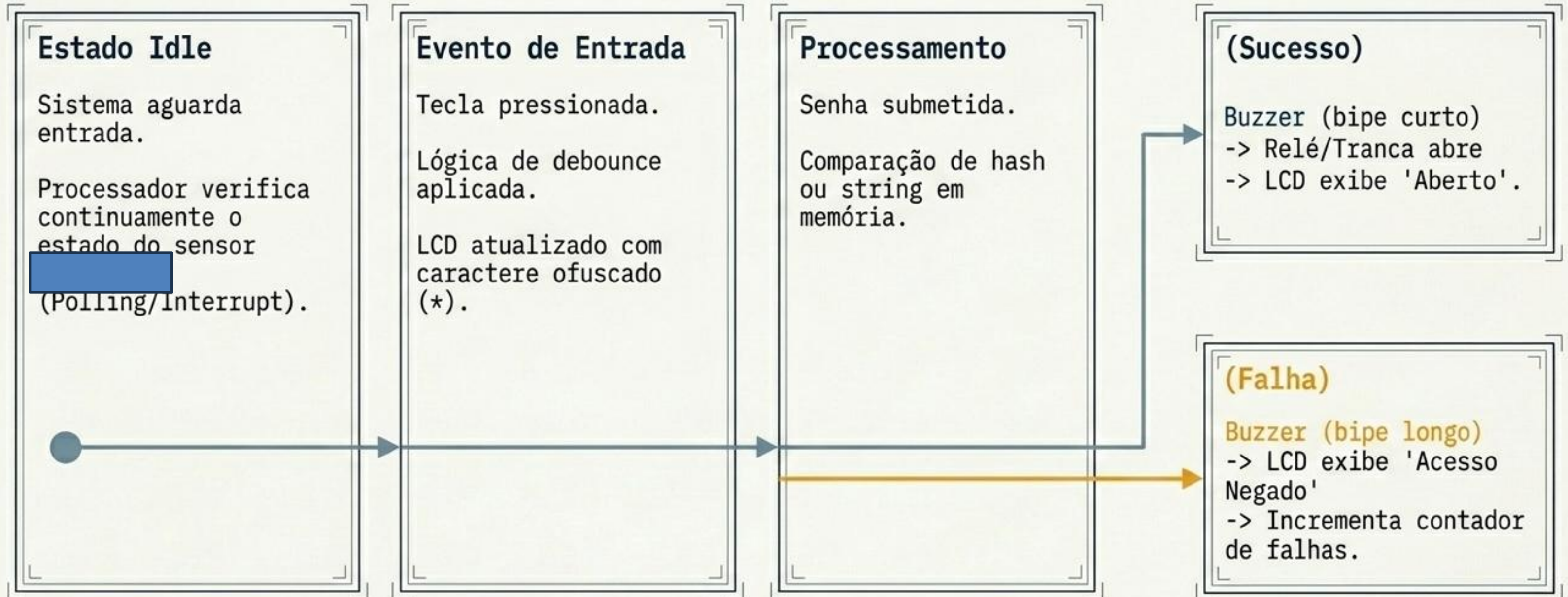
Arquitetura de Hardware e Decisões de Projeto



Por que utilizar I2C para o LCD?

- 1. Eficiência de Pinos:** Reduz drasticamente a utilização de pinos GPIO (apenas 2 pinos: SDA e SCL vs. 6+ no modo paralelo).
- 2. Delegação de Carga:** Delega a temporização do display para o módulo I2C interno, liberando a CPU para o polling do teclado.

Arquitetura de Software: Fluxo de Estado Não-Bloqueante



Deep Dive: Suporte nativo I2C no Raspberry Pi 3

Nível de Aplicação

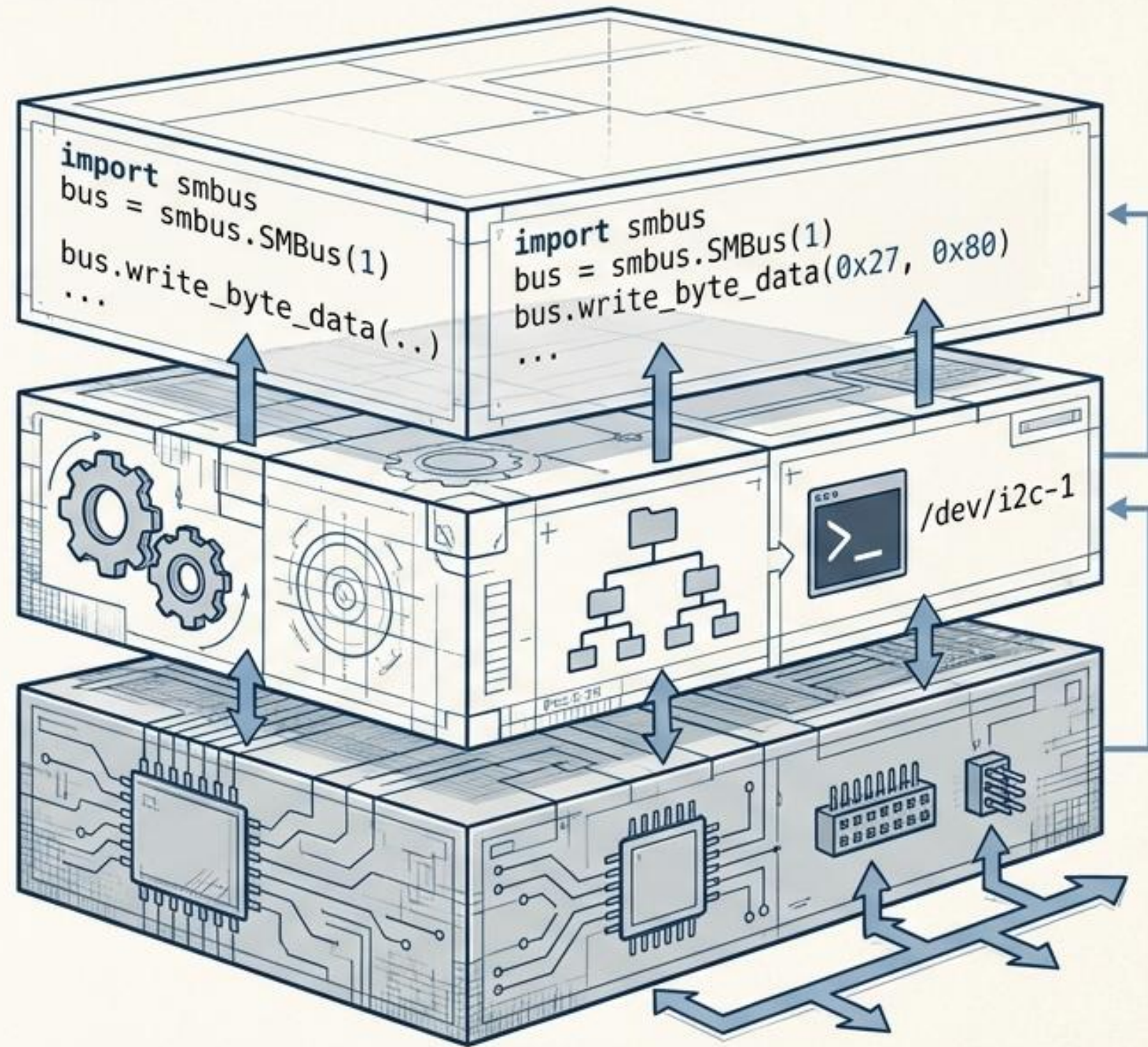
Bibliotecas de alto nível (ex: smbus em Python) abstraem a sinalização de clock e controle de bits, permitindo o envio direto de bytes para o endereço hexadecimal do LCD.

Nível de Sistema Operacional (Linux Kernel)

O suporte é intermediado pelo Device Tree do Linux e módulos de kernel (i2c-dev). O SO trata o barramento como um arquivo de dispositivo (/dev/i2c-1).

Nível de Hardware (SoC Broadcom BCM2837)

O RPi3 possui controladores I2C (Broadcom Serial Controllers) integrados fisicamente no silício do chip ARM.



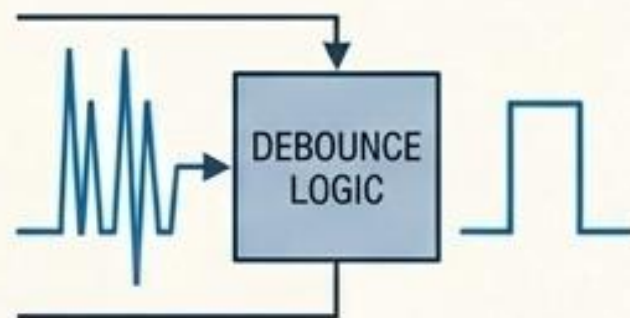
Exigência: Documente as referências ABNT utilizadas para esta pesquisa no relatório final.

A Fundação: Controle e Isolamento de Componentes

Regra de Ouro: Nunca integre um componente que não passou em seu próprio teste unitário.

Teclado: Validar Debouncing

A leitura de uma tecla gera apenas um evento? Pinos flutuantes foram evitados com resistores pull-up/pull-down (internos ou externos)?



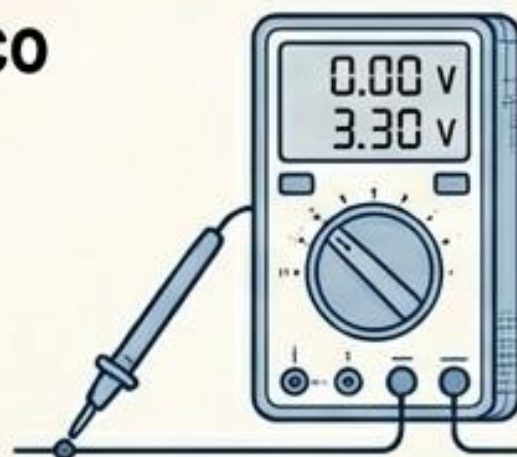
LCD via I2C: Validar Endereçamento

O comando `i2cdetect -y 1` encontra o display na rede?
O script isolado imprime 'Hello World'?

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00:																
10:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
20:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
30:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
40:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
50:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
60:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
70:	--	--	--	--	--	--	--	--	--	27	--	--	--	--	--	--

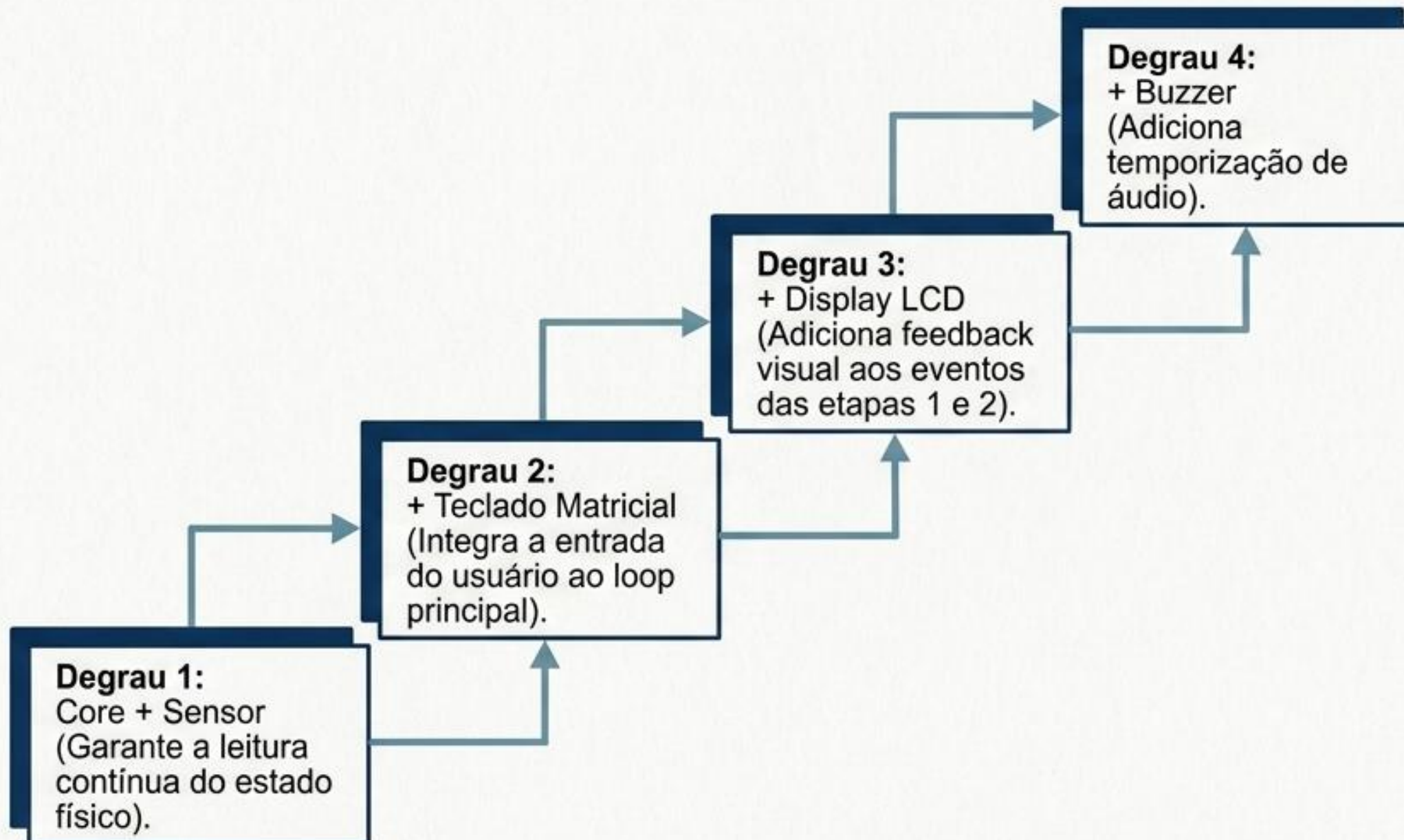
Sensor: Validar Nível Lógico

A transição de estado (Aberto/Fechado) reflete corretamente 0V e 3.3V no pino GPIO?



Buzzer: Validar Acionamento

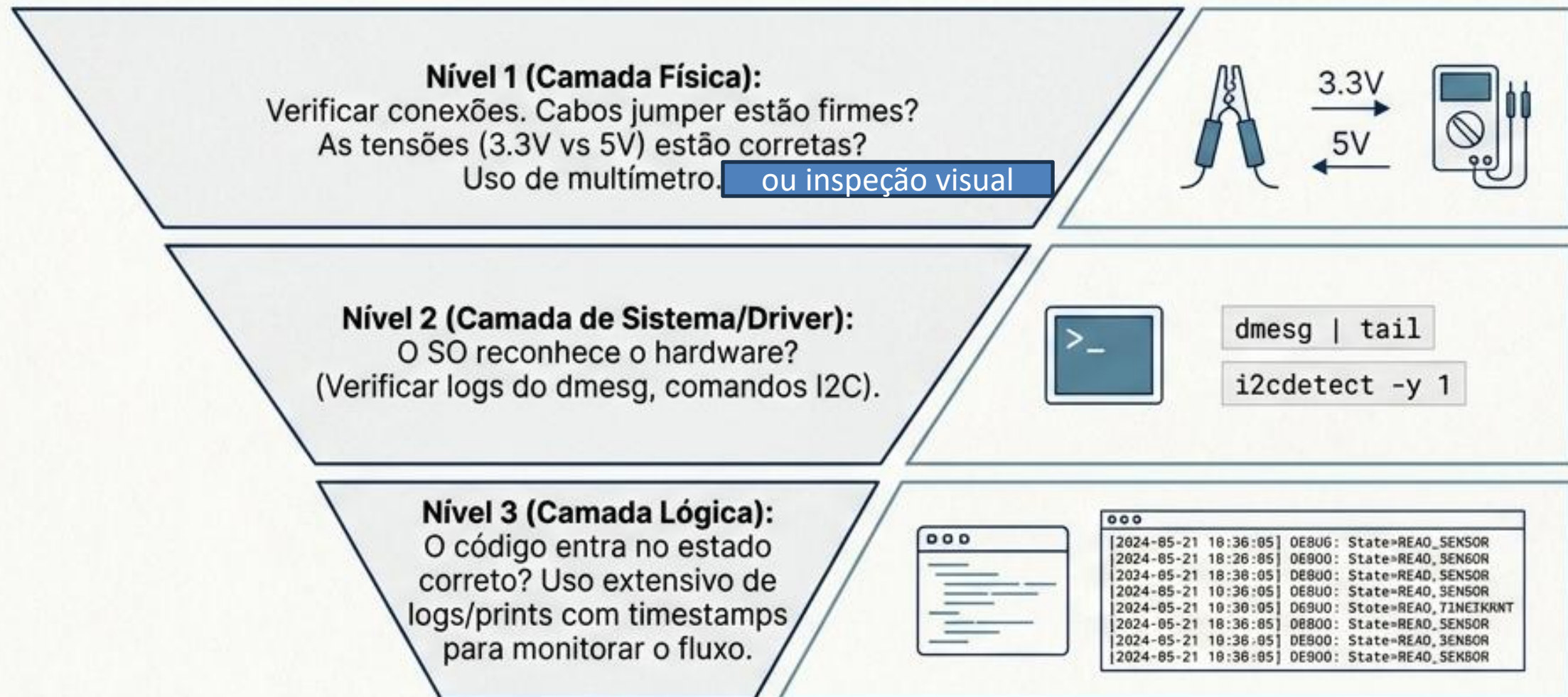
O Desafio de Integração: Uma Abordagem Iterativa



⚠️ Desafios Principais (Ameaças à Integração)

- Conflito de Recursos: Pinos GPIO definidos incorretamente em múltiplos scripts.
- Código Bloqueante: Uso de funções como `sleep()` para o Buzzer que acabam congelando a varredura do teclado ou ignorando a abertura do sensor.
- ⚠️ Gerenciamento de Estado: O sistema se perde se múltiplas coisas acontecerem ao mesmo tempo.

Práticas de Depuração: O Funil de Isolamento de Falhas



Registro no Relatório

O que deu errado? Como foi corrigido?
Exemplo: "O teclado registrava múltiplas entradas (bouncing). Refinamos o método adicionando um delay de 50ms na varredura."
Erros documentados são **evidências de engenharia sólida**.

Desafio Avançado: Análise de Vulnerabilidades e Ameaças

Motivação do Atacante: Obter acesso físico não autorizado ao ambiente protegido pela fechadura.



Vetor de Ataque Exemplificado: "Tampering" Físico e Spoofing de Sensor

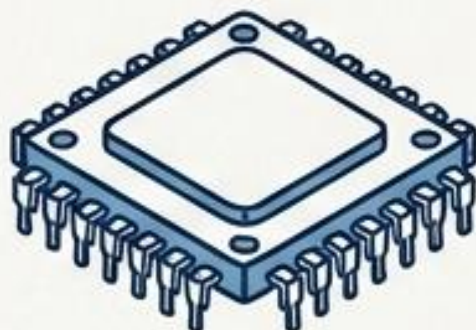
Passo 1:	O atacante acessa a fiação exposta do sensor 
Passo 2:	O atacante insere um jumper ou imã externo, forçando o estado lógico "FECHADO" no GPIO, independentemente da porta real. GPIO SET: STATE=CLOSED (FORCED)
Passo 3:	A porta é fisicamente arrombada. 

Impacto: O Raspberry Pi 3 continua processando o estado "trancado" e o alarme (buzzer) nunca é acionado, anulando a segurança lógica do sistema.

Criptografia e Restrições de Hardware: RPi3 vs. ESP32

Como a Criptografia Protege? Armazenamento de senhas em hashes (ex: SHA-256) em vez de texto plano. Comunicação criptografada caso a fechadura se conecte a uma rede IoT.

Raspberry Pi 3 (ARM Cortex-A53 + Linux)



ESP32 (Microcontrolador)



Pró:

Processamento massivo. Suporta bibliotecas completas (OpenSSL, Python hashlib).

Dificuldade:

Vulnerável a ataques de temporização (timing attacks) e injeção devido ao agendador de processos (scheduler) imprevisível do Linux e à vasta superfície de ataque de um SO completo.

Pró:

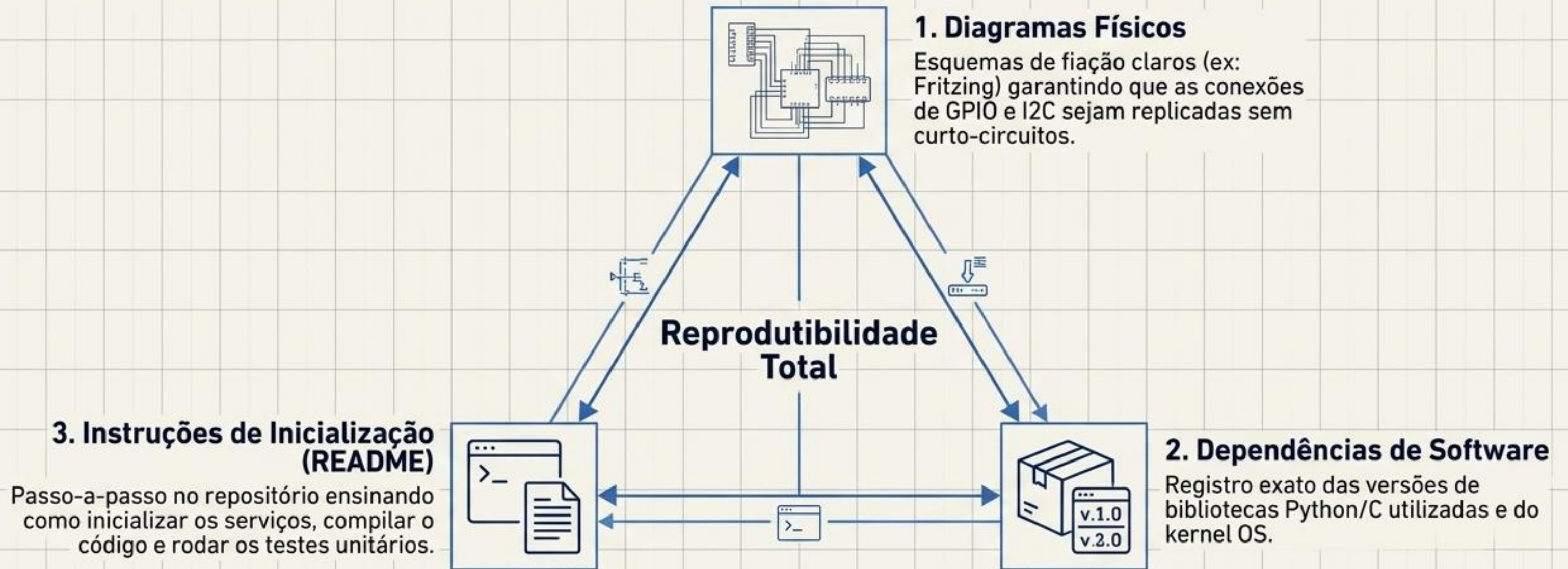
Possui aceleradores criptográficos em hardware (AES, SHA). Execução determinística (RTOS) ideal para evitar timing attacks.

Dificuldade:

Recursos limitados de RAM. Gerenciamento de chaves públicas/certificados é complexo sem um SO completo.

A Missão não termina no código: Reprodutibilidade

A Importância da Documentação: Como a documentação de um projeto suporta sua reprodutibilidade por terceiros?



Nota Histórica: Assim como o código original do Apollo 11 sobrevive e pode ser estudado hoje graças à digitalização meticulosa, seu projeto deve ser claro o suficiente para ser replicado por outro engenheiro.

Check-list do Relatório da Missão (Aula 10)

☐	Formatação: PDF, Máximo de 10 páginas, Fonte Arial 12.
☐	Requisitos & Arquitetura: Matriz de testes preenchida e diagramas de bloco/fluxo documentados e justificados.
☐	Método e Depuração: Descrição do plano de integração. O que deu certo? O que falhou e como foi corrigido? (Lições Aprendidas).
☐	Respostas Técnicas: Questões sobre I2C, integração, depuração e segurança respondidas com citações ABNT baseadas nas leituras pré-aula.
☐	Repositório Público: Link para o GitHub (obrigatório, conforme Instruções da Disciplina) contendo todo o código desenvolvido.